

Name: Y.Veerendar Reddy

Reg.No:- 192325102

Subject Code/Name: MLA0305-Reinforcement Learning

EXPERIMENT - 11

Policy Gradient Implementation Using REINFORCE Algorithm

Aim

To implement the REINFORCE Policy Gradient algorithm using TensorFlow and Keras and train an agent to learn a suitable policy in the CartPole environment.

Procedure

1. Create the CartPole environment.
2. Build a neural-network policy using TensorFlow and Keras.
3. Use the network to calculate action probabilities.
4. Select actions according to the probabilities.
5. Store states, actions, and rewards for each episode.
6. Calculate the discounted return for each step.
7. Update the policy using the REINFORCE policy-gradient method.
8. Repeat the process for several episodes.
9. Plot the rewards to evaluate learning performance.

Code:

```
# =====  
# EXPERIMENT 11: POLICY GRADIENT (REINFORCE)  
# =====  
  
import numpy as np  
import tensorflow as tf  
import gymnasium as gym  
import matplotlib.pyplot as plt  
  
# Reproducibility  
np.random.seed(42)  
tf.random.set_seed(42)
```

```

# -----
# Environment
# -----

env = gym.make("CartPole-v1")
state_size = env.observation_space.shape[0]
action_size = int(env.action_space.n)


# -----
# Policy Network
# -----

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(state_size,)),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(action_size, activation="softmax")
])

optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)


# -----
# Training (REINFORCE)
# -----

episodes = 100
gamma = 0.99
reward_history = []

for episode in range(episodes):
    state, _ = env.reset()
    states, actions, rewards = [], [], []
    done = False

    # Generate episode
    while not done:
        state_input = np.array([state], dtype=np.float32)
        probs = model(state_input, training=False).numpy()[0]

```

```

action = np.random.choice(action_size, p=probs)

next_state, reward, terminated, truncated, _ = env.step(action)
done = terminated or truncated

states.append(state)
actions.append(action)
rewards.append(reward)
state = next_state

# Discounted returns
returns, G = [], 0
for r in reversed(rewards):
    G = r + gamma * G
    returns.insert(0, G)
returns = np.array(returns, dtype=np.float32)
returns = (returns - np.mean(returns)) / (np.std(returns) + 1e-8)

states_array = np.array(states, dtype=np.float32)
actions_array = np.array(actions, dtype=np.int32)

# Policy update
with tf.GradientTape() as tape:
    probs = model(states_array, training=True)
    action_one_hot = tf.one_hot(actions_array, action_size)
    selected_probs = tf.reduce_sum(probs * action_one_hot, axis=1)
    log_probs = tf.math.log(selected_probs + 1e-8)
    loss = -tf.reduce_mean(log_probs * returns)

grads = tape.gradient(loss, model.trainable_variables)
optimizer.apply_gradients(zip(grads, model.trainable_variables))

total_reward = sum(rewards)
reward_history.append(total_reward)

```

```

print(f"Episode {episode+1:3d} | Reward: {total_reward:3d}")

env.close()

# -----
# Results
# -----

print("\n" + "="*50)
print("Average Reward:", round(np.mean(reward_history), 2))
print("Best Reward:", max(reward_history))

# -----
# Visualization
# -----

plt.figure(figsize=(10,5))
plt.plot(reward_history, label="Episode Reward")
plt.title("REINFORCE Policy Gradient Learning")
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.legend()
plt.grid(True)
plt.show()

# Moving average
window = 10
if len(reward_history) >= window:
    moving_avg = np.convolve(reward_history, np.ones(window)/window, mode="valid")
    plt.figure(figsize=(10,5))
    plt.plot(moving_avg, label="10-Episode Moving Average")
    plt.title("REINFORCE Moving Average Reward")
    plt.xlabel("Episode")
    plt.ylabel("Average Reward")
    plt.legend()
    plt.grid(True)
    plt.show()

```

OUTPUT:-

Episode 1 | Reward: 44

Episode 2 | Reward: 15

Episode 3 | Reward: 13

Episode 4 | Reward: 26

Episode 5 | Reward: 15

Episode 6 | Reward: 10

Episode 7 | Reward: 16

Episode 8 | Reward: 25

Episode 9 | Reward: 27

Episode 10 | Reward: 14

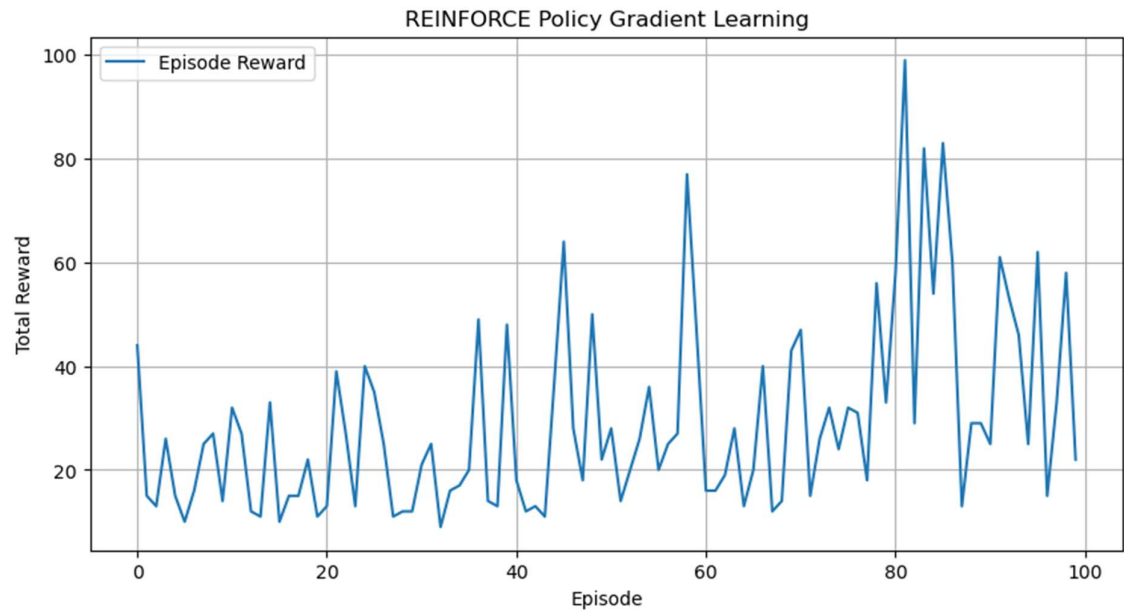
=====

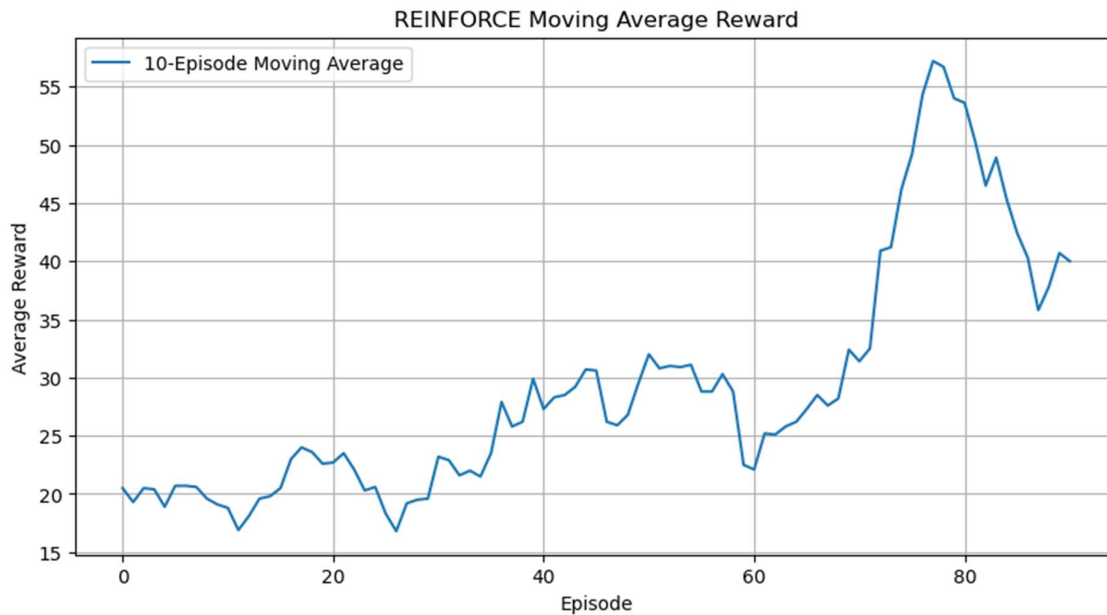
REINFORCE RESULTS

=====

Average Reward: 29.16

Best Reward: 99





Visualization

The program generates:

1. **Episode Reward Graph** – shows the reward obtained by the agent in every episode.
2. **Moving Average Reward Graph** – shows the average performance over 10 consecutive episodes and helps observe the learning trend.

Summary

The REINFORCE policy-gradient algorithm was implemented using a neural-network policy. The agent selected actions according to the learned probability distribution and updated the policy using discounted episode rewards.

Result

Thus, the REINFORCE Policy Gradient algorithm was successfully implemented using TensorFlow and Keras, and the agent's learning performance was evaluated through reward visualizations.